

Weekly Engineering Assignment 4 - Group 3 - S23

Driveway Cost Table

Section 1: Basic Functions

1. Simple conversion functions

I will make several functions to convert unit. These functions will be used in the main function and will be basic and easy to use. It will help reduce unnecessary code usage and increase readability. This little piece of function is going to produce a big result.

```
In [10]: # Your function to convert meters to inches
def meters_to_inches(meter):
    return (meter * 39.3701)
```

```
In [11]: # Your function to convert inches to meters
def inches_to_meters(inches):
    return (inches / 39.3701)
```

```
In [12]: values = [0, 1, 1.7]
for i in values:
    print("The vvalue is %.2f, meter to inch is %f inches, inch to meter is %f meters"
```

```
The vvalue is 0.00, meter to inch is 0.000000 inches, inch to meter is 0.000000 meter
s
The vvalue is 1.00, meter to inch is 39.370100 inches, inch to meter is 0.025400 mete
rs
The vvalue is 1.70, meter to inch is 66.929170 inches, inch to meter is 0.043180 mete
rs
```

2. Four more Conversion Functions

I will write new for functions, the functions's name will be intuitive and simple.

```
In [13]: # Your functions that can convert square/cubic meters to square/cubic inches or
# square/cubic inches to square/cubic meters
def square_meters_to_square_inches(meters):
    return meters * (39.3701 * 39.3701)
def square_inches_to_square_meters(inches):
    return inches / (39.3701 * 39.3701)
def cubic_meters_to_cubic_inches(meters):
    return meters * (39.3701 * 39.3701 * 39.3701)
def cubic_inches_to_cubic_meters(inches):
    return inches / (39.3701 * 39.3701 * 39.3701)
```

```
In [14]: # Your three test cases for each of the four new functions, confirming the correctness
for i in values:
    print("The value of i is : %.2f" %i)
    print("square meters %.2f to square inches = %.2f" %(i,square_meters_to_square_inches(i)))
    print("square inches %.2f to square meters = %.2f" %(i,square_inches_to_square_meters(i)))
    print("cubic meters %.2f to cubic inches = %.2f" %(i,cubic_meters_to_cubic_inches(i)))
    print("cubic inches %.2f to cubic meters = %.2f" %(i,cubic_inches_to_cubic_meters(i)))
```

```
The value of i is : 0.00
square meters 0.00 to square inches = 0.00
square inches 0.00 to square meters = 0.000000
cubic meters 0.00 to cubic inches = 0.00
cubic inches 0.00 to cubic meters = 0.000000
The value of i is : 1.00
square meters 1.00 to square inches = 1550.00
square inches 1.00 to square meters = 0.000645
cubic meters 1.00 to cubic inches = 61023.84
cubic inches 1.00 to cubic meters = 0.000016
The value of i is : 1.70
square meters 1.70 to square inches = 2635.01
square inches 1.70 to square meters = 0.001097
cubic meters 1.70 to cubic inches = 103740.53
cubic inches 1.70 to cubic meters = 0.000028
```

3. Write a controller function

In now, I have 6 functions and I will make a function that includes all of them. And, the string parameter will be used to determine which function to call.

```
In [15]: # Your controller function
def controller(event, values):
    if event == "inches to meters":
        return inches_to_meters(values)
    elif event == "meters to inches":
        return meters_to_inches(values)
    elif event == "square meters to square inches":
        return square_meters_to_square_inches(values)
    elif event == "square inches to square meters":
        return square_inches_to_square_meters(values)
    elif event == "cubic meters to cubic inches":
        return cubic_meters_to_cubic_inches(values)
    elif event == "cubic inches to cubic meters":
        return cubic_inches_to_cubic_meters(values)
    else:
        print("Invalid event, you can only use the following events: inches to meters, meters to inches, square meters to square inches, square inches to square meters, cubic meters to cubic inches, cubic inches to cubic meters")
        return None
```

```
In [16]: # Your tests to confirm that it works as designed.
print("1 inches to meters: %f" %controller("inches to meters", 1))
print("1 meters to inches: %f" %controller("meters to inches", 1))
print("1 square meters to square inches: %f" %controller("square meters to square inches", 1))
print("1 square inches to square meters: %f" %controller("square inches to square meters", 1))
print("1 cubic meters to cubic inches: %f" %controller("cubic meters to cubic inches", 1))
print("1 cubic inches to cubic meters: %f" %controller("cubic inches to cubic meters", 1))
```

```

1 inches to meters: 0.025400
1 meters to inches: 39.370100
1 square meters to square inches: 1550.004774
1 square inches to square meters: 0.000645
1 cubic meters to cubic inches: 61023.842953
1 cubic_inches_to_cubic_meters: 0.000016

```

4. Convert values from a file

I will read the file and divide 3 elements into 3 parts: current unit, converted unit, and value. and the combination of current unit and converted unit will be used to determine the statement. That statement will call the appropriate function.

```

In [33]: def determine_type(current_unit, converted_unit):
          dic = {("inches", "meters") : "inches to meters", ("meters", "inches") : "meters to inches"}
          return dic[(current_unit, converted_unit)]
data_lst = []
with open("values.txt", "r") as infile:
    lines = infile.readlines()
    for line in lines:
        previous_unit, converted_unit, value = line.split()
        value = float(value)
        print("%s %f convert to %s: " % (previous_unit, value, converted_unit), end = " ")
        print(controller(determine_type(previous_unit, converted_unit), value)) # check
        data_lst.append((converted_unit, controller(determine_type(previous_unit, converted_unit), value)))
    print("-----" * 2)
with open("converted_values.txt", "w") as outfile:
    for i in data_lst:
        outfile.write(str(i) + "\n") # write the data to the file
    print(i)

```

```

inches 36.000000 convert to meters: 0.9143995062242666
meters 2.000000 convert to inches: 78.7402
square_inches 148.200000 convert to square_meters: 0.09561260873835467
square_meters 12.100000 convert to square_inches: 18755.057765521
cubic_inches 1728.000000 convert to cubic_meters: 0.02831680071875806
cubic_meters 1.000000 convert to cubic_inches: 61023.84295325111
-----
('meters', 0.9143995062242666)
('inches', 78.7402)
('square_meters', 0.09561260873835467)
('square_inches', 18755.057765521)
('cubic_meters', 0.02831680071875806)
('cubic_inches', 61023.84295325111)

```

Section 2: Convert Lists of Value

The controller function will examine the second parameter whether it is a single value or list of values. And, I will add the function that create a new list and return it. And, The code is too long and not easy to read. I will organize the code in a way that more simple and easy to read.

```

In [37]: # Your controller function
def controller(event, values):

```

```

event_mapping = {
    "inches to meters": inches_to_meters,
    "meters to inches": meters_to_inches,
    "square meters to square inches": square_meters_to_square_inches,
    "square inches to square meters": square_inches_to_square_meters,
    "cubic meters to cubic inches": cubic_meters_to_cubic_inches,
    "cubic inches to cubic meters": cubic_inches_to_cubic_meters,
}

if event not in event_mapping:
    print("Invalid event, you can only use the following events: inches to meters,")
    return None

if isinstance(values, (int, float)):
    return event_mapping[event](values)

elif isinstance(values, list):
    return [event_mapping[event](value) for value in values]

# test
test_lst = [0, 1, 1.75]

for event in ["inches to meters", "meters to inches", "square meters to square inches"]:
    print(f"Testing {event} with values {test_lst}:")
    result = controller(event, test_lst)
    print(f"Result: {result}\n")

```

Testing inches to meters with values [0, 1, 1.75]:
 Result: [0.0, 0.025399986284007407, 0.04444997599701296]

Testing meters to inches with values [0, 1, 1.75]:
 Result: [0.0, 39.3701, 68.897675]

Testing square meters to square inches with values [0, 1, 1.75]:
 Result: [0.0, 1550.0047740100001, 2712.5083545175003]

Testing square inches to square meters with values [0, 1, 1.75]:
 Result: [0.0, 0.0006451593032277644, 0.0011290287806485877]

Testing cubic meters to cubic inches with values [0, 1, 1.75]:
 Result: [0.0, 61023.84295325111, 106791.72516818944]

Testing cubic inches to cubic meters with values [0, 1, 1.75]:
 Result: [0.0, 1.638703745298499e-05, 2.8677315542723732e-05]

Honestly, I don't know what's different from the previous question and code. But I'll try. I will solve these by functioning more simply. This function process_file will take all the lines in the file and determine the unit. Also, create the converted value and write it to the output_file.

```

In [51]: def process_file(input_file, output_file): #input file is values.txt, and output file
        with open(input_file, 'r') as infile:
            lines = infile.readlines()

        converted_values = []
        for line in lines:
            previous_unit, converted_unit, value = line.split()

```

```

        value = float(value)
        converted_values.append((converted_unit, controller(determine_type(previous_ur
with open(output_file, "w") as outfile:
    for i in converted_values:
        outfile.write(str(i) + "\n") # write the data to the file

# Print the contents of the output file
with open(output_file, 'r') as outfile:
    lines = outfile.readlines()
    for line in lines:
        print(line.rstrip())

process_file('values.txt', 'converted_values2.txt')

```

```

('meters', 0.9143995062242666)
('inches', 78.7402)
('square_meters', 0.09561260873835467)
('square_inches', 18755.057765521)
('cubic_meters', 0.02831680071875806)
('cubic_inches', 61023.84295325111)

```

Section 3: Convert lists of lists of value

My existing controller can solve a single list, but this function cannot solve the data of the list in list structure yet. This will be supplemented by using recursive function.

```

In [96]: event_mapping = {
    "inches to meters": inches_to_meters,
    "meters to inches": meters_to_inches,
    "square meters to square inches": square_meters_to_square_inches,
    "square inches to square meters": square_inches_to_square_meters,
    "cubic meters to cubic inches": cubic_meters_to_cubic_inches,
    "cubic inches to cubic meters": cubic_inches_to_cubic_meters,
}

def controller(event, values):
    data = []
    if event not in event_mapping:
        print("Invalid event, you can only use the following events: inches to meters,
        return None

    if isinstance(values, (int, float)):
        return event_mapping[event](values)

    elif isinstance(values, list):
        return [controller(event, i) for i in values]

ex1 = 2
ex2 = [2]
ex3 = [2, 3]
ex4 = [2, [2, 3]]
ex5 = [1, [1, 2, 3], [1, [1, 2, 3]]]
order = "inches to meters"

```

```
print(controller(order, ex1))
print(controller(order, ex2))
print(controller(order, ex3))
print(controller(order, ex4))
print(controller(order, ex5))
```

```
0.050799972568014815
[0.050799972568014815]
[0.050799972568014815, 0.07619995885202221]
[0.050799972568014815, [0.050799972568014815, 0.07619995885202221]]
[0.025399986284007407, [0.025399986284007407, 0.050799972568014815, 0.07619995885202221], [0.025399986284007407, [0.025399986284007407, 0.050799972568014815, 0.07619995885202221]]]
```

The purpose of this assignment is largely divided into three categories. 1. Defintion the function. 2. read the file and write the file. 3. Flexible functions that are not constrained by datatypes. I declared the fundamental functions, and I put them together. I read the file and converted the number using the function. I also saved the results to a new file. And I put all of these processes together as a function.(process_file) In section 3, to transform the nested list, I used the recursive function, but I'm not sure if this is the derived answer. If there is any other good way, please comment.